

In The Name of God

Type of Gradient Decent

Name: Hossein Heydari

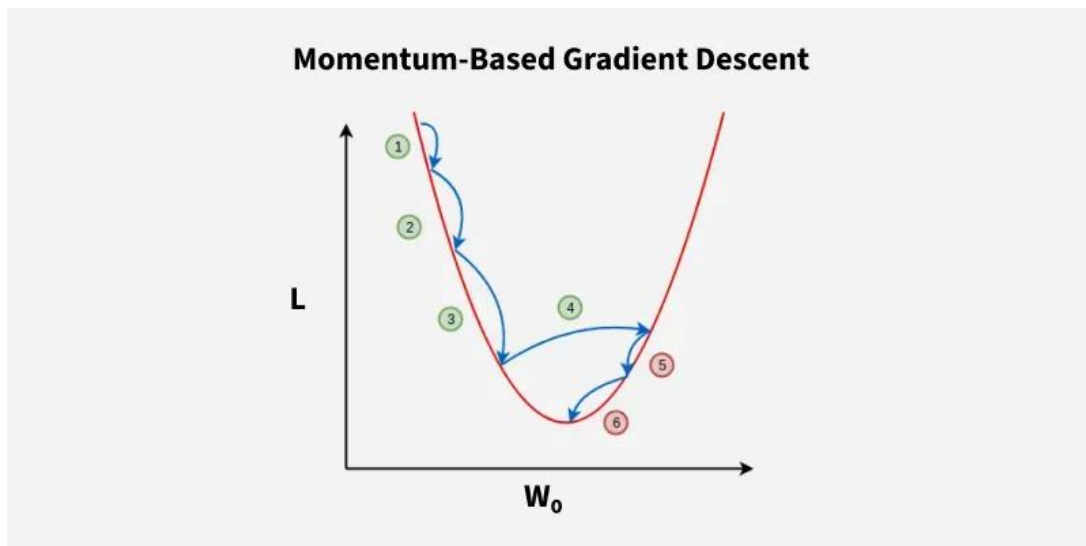
Date: June 2026

a) Momentum-Based Gradient Descent:

Momentum-based optimizers improve standard gradient descent by adding a momentum term that helps move more efficiently across the loss surface.

Momentum is inspired by physics, where movement depends on both current force and past velocity. In optimization, it helps smooth the learning process by incorporating past gradients, leading to faster and more stable convergence.

- Uses past gradients to accelerate learning
- Reduces oscillations during training
- Helps achieve faster convergence
- Improves performance in deep networks and large datasets



Formula:

$$v_t + 1 = \beta v_t + 1 + \beta \nabla L w_t$$

$$w_t + 1 = w_t - n v_t + 1$$

Where:

- v_t : is the velocity i.e., a running average of gradients
- β : is the momentum factor, typically a value between 0 and 1 (often around 0.9)
- $\nabla L w_t$: is the current gradient of the loss function
- n : is the learning rate

Understanding Hyperparameters:

- Learning Rate(n): The learning rate determines the size of the step taken during each update. It plays a crucial role in both standard gradient descent and momentum-based optimizers.
- Momentum Factor(β): This controls how much of the past gradients are remembered in the current update. A value close to 1 means the optimizer will have more inertia while a value closer to 0 means less reliance on past gradients.

Working of the Algorithm?

1. Velocity Update: The velocity v_t is updated by considering both the previous velocity which represents the momentum and the current gradient. The momentum factor β controls the contribution of the previous velocity to the current update.

2. Weight Update: The weights are updated using the velocity $v_t + 1$ which is a weighted average of the past gradients and the current gradient.

Where is momentum-based gradient descent used?

- One of the most important applications of momentum is in training neural networks:
 - Multilayer Perceptron (MLP)
 - Convolutional Neural Networks (CNN)
 - Recurrent Neural Networks (RNN)

Reason for use:

- The number of parameters is large.
- Gradients may fluctuate.
- Convergence with conventional Gradient Descent is slow.

- Cost function with narrow valleys (Ravines):

Sometimes the cost function has a shape similar to a narrow valley.

In this case:

Normal Gradient Descent:

- It goes left once.
- It goes right once.
- It takes a zigzag path.

But Momentum:

- Maintains the previous direction of movement.

- Reduces lateral oscillations.
- It moves faster towards the minimum.

➤ Large datasets:

Suppose:

- A. 1 million samples
- B. 100 features

In this situation, it is possible that they are novel.

Momentum makes:

- 1) The impact of noise less.
- 2) The software moves.
- 3) The training process becomes more stable.

➤ Modern optimization algorithms:

Many popular optimizers use the idea of momentum, such as:

- SGD with Momentum
- Nesterov Accelerated Gradient
- Adam
- RMSprop

b) Nesterov Accelerated Gradient (NAG):

Nesterov Accelerated Gradient is a refined version of momentum-based optimization, designed to further smooth and speed up convergence. It does so by looking ahead at the future gradient and adjusting accordingly, which helps reduce overshooting and improves stability.

In simple terms, NAG calculates the gradient at a “look-ahead” position rather than the current position. This look-ahead step is what makes NAG more effective at navigating tricky error surfaces, especially in non-convex optimization problems.

How NAG Works:

1) Momentum Update:

Traditional momentum calculates an update based on the current gradient and a weighted sum of past gradients, creating a “velocity” term that guides the update direction.

2) NAG Update:

Instead of calculating the gradient at the current point, NAG takes a look-ahead step using the momentum term. It then computes the gradient at this new look-ahead position and adjusts the update based on this gradient. This approach makes it easier to predict when the optimizer is likely to overshoot, allowing for more controlled updates.

Example:

Imagine rolling a ball down a hill. With simple momentum, you only know the direction you’re moving in and rely on past speed to guide you. But with NAG, it’s like you’re peeking a few steps ahead to see if you’re about to overshoot. This helps you adjust the speed to avoid missing the target.

Formulation of NAG:

1. Calculate the Look-Ahead Point: First, we determine where we’d be if we took a step forward with just the momentum.

$$\mathcal{W}_{look-ahead} = \omega - \beta \cdot v_t - 1$$

2. Calculate the Gradient at the Look-Ahead Point: We compute the gradient at this position:

$$gradient = \nabla f(\mathcal{W}_{look-ahead})$$

3. **Update the Velocity Term:** The velocity term is adjusted based on the look-ahead gradient:

$$v_t = \beta \cdot v_{t-1} + \eta \cdot gradient$$

4. **Update the Weights:** Finally, the weights are updated based on the look-ahead velocity:

$$\mathcal{W} = \mathcal{W} - v_t$$

Advantages of NAG:

1. **Reduced Oscillations:** By peeking ahead, NAG reduces oscillations around the minimum, which can be especially useful in error surfaces with multiple peaks and valleys.
2. **Faster Convergence:** NAG often converges faster than simple momentum-based methods because it reduces unnecessary steps around the optimum.



```
from tensorflow.keras.optimizers import SGD
optimizer = SGD(learning_rate=0.01, momentum=0.9,
nesterov=True)
```

c) AdaGrad

Adagrad is an optimization method that adapts the learning rate for each parameter based on past gradients, improving learning for features with different frequencies.

- Adjusts learning rate individually for each parameter
- Uses accumulated past gradients to scale updates
- Works well for sparse data and varying feature magnitudes
- Reduces learning rate over time for frequently updated parameters

How work:

- 1) Initialization: Adagrad begins by initializing the parameter values randomly, just like other optimization algorithms. Additionally, it initializes a running sum of squared gradients for each parameter which will track the gradients over time.
- 2) Gradient Calculation: For each training step, the gradient of the loss function with respect to the model's parameters is calculated, just like in standard gradient descent.
- 3) Adaptive Learning Rate: Adagrad adjusts the learning rate for each parameter based on the accumulated sum of squared gradients, instead of using a fixed rate.

Formula:

$$\eta_t = \frac{\eta}{\sqrt{G_t + \epsilon}}$$

Where:

- η is the global learning rate (a small constant value)
- G_t is the sum of squared gradients for a given parameter up to time step t
- ϵ is a small value added to avoid division by zero (often set to $1e - 8$)
- As $\sqrt{G_t + \epsilon}$ increases, the learning rate decreases over time

4) Parameter Update: The model's parameters are updated by subtracting the product of the adaptive learning rate and the gradient at each step:

$$\theta_{t+1} = \theta_t - lr_t \cdot \nabla_{\theta}$$

Where:

- θ_t is the current parameter
- $\nabla_{\theta} J(\theta)$ is the gradient of the loss function with respect to the parameter

Use Cases of Adagrad:

- Works well for sparse data such as **NLP** and recommender systems
- Useful when features have different importance or frequency
- Suitable for tasks that prefer stable learning over very fast convergence
- May not perform well when a consistent learning rate is needed
- In such cases, optimizers like RMSProp or Adam are often preferred

```

1 import tensorflow as tf
2 from tensorflow.keras.datasets import mnist
3 from tensorflow.keras.utils import to_categorical
4
5 (x_train, y_train), (x_test, y_test) = mnist.load_data()
6
7 x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
8 x_test = x_test.reshape(-1, 784).astype('float32') / 255.0
9
10 model = tf.keras.Sequential([
11     tf.keras.layers.Dense(64, activation='relu', input_shape=(784,)),
12     tf.keras.layers.Dense(10, activation='softmax')
13 ])
14
15 model.compile(optimizer=tf.keras.optimizers.Adagrad(learning_rate=0.01),
16               loss='sparse_categorical_crossentropy',
17               metrics=['accuracy'])
18
19 model.fit(x_train, y_train, epochs=5)

```

d) **RMSProp:**

RMSProp was developed to overcome limitations of earlier methods like SGD and Adagrad by improving learning rate adaptation.

- SGD uses a constant learning rate, which can be inefficient
- Adagrad decreases the learning rate too quickly over time
- RMSProp uses a moving average of squared gradients to adapt learning rates
- Maintains a balance between fast convergence and training stability
- Widely used in deep learning for efficient optimization

How work:

- RMSProp works by maintaining a moving average of squared gradients to normalize updates and adapt the learning rate for each parameter.
- Keeps a moving average of squared gradients
- Prevents learning rate from becoming too small (issue in Adagrad)
- Scales updates appropriately for each parameter
- Handles non-stationary objectives effectively

Formula:

1. Compute the gradient g_t at time step t :

$$g_t = \nabla \theta$$

2. Update the moving average of squared gradients:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)$$

3. Update the parameter θ using the adjusted learning rate:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}}$$

Where:

- γ is the decay rate
- η is the learning rate
- ϵ is a small constant added for numerical stability

```
1 tf.keras.optimizers.RMSprop(learning_rate=0.001,  
2 rho=0.9)
```

e) Adam:

Adam (Adaptive Moment Estimation) optimizer combines the advantages of Momentum and RMSprop techniques to adjust learning rates during training. It works well with large datasets and complex models because it uses memory efficiently and adapts the learning rate for each parameter automatically.

How Work:

- First moment (mean) estimate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \left(\frac{\partial L}{\partial w_t} \right)$$

- Second moment (variance) estimate:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

- Bias correction:

Since both m_t and v_t are initialized at zero, they tend to be biased toward zero, especially during the initial steps. To correct this bias, Adam computes the bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

- Final weight update: The weights are then updated as:

$$w_{t+1} = w_t - \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \alpha$$

Key Parameters:

- α : The learning rate or step size (default is 0.001)
- β_1 and β_2 : Decay rates for the moving averages of the gradient and squared gradient, typically set to $\beta_1 = 0.9$ and $\beta_2 = 0.999$
- ϵ : A small positive constant (e.g., 10^{-8}) used to avoid division by zero when computing the final update

Implementation:

```
1 from tensorflow.keras.optimizers import  
  Adam  
2 adam_optimizer = Adam(learning_rate=0.001)  
3
```

f) AdamW:

In machine learning, a penalty technique called L2 Regularization is commonly used to prevent overfitting and control model complexity. In traditional optimizers like SGD, applying L2 Regularization is mathematically equivalent to a technique called Weight Decay. In this technique, a small amount of the weights is decreased at each step to prevent the weights from becoming too large.

But when the developers designed the Adam algorithm, they incorporated L2 Regularization directly into the gradient calculation formula. Due to Adam's specific mechanism (which uses an Adaptive Learning Rate), this combination caused the penalty applied to the weights to be affected by the moments.

Problem Summary: In Adam, weights with large gradients were penalized less than required and weights with small gradients were penalized more than required. This interference made the generalization property of models trained with Adam not as good as SGD.

In 2017, researchers (**Loshchilov** and **Hutter**) introduced the AdamW algorithm (where W stands for Weight Decay). Their idea was very simple but crucial: to separate the weight decay calculations from the gradient calculations.

In AdamW, the optimization formula was changed to apply the weight penalty independently after calculating the adaptive learning rate. If we want to see the formula for updating the weights (w) in simple mathematical language:

$$\theta_{t+1} = \theta_t - \eta_t \cdot \text{Adam}(\text{gradients}) - \eta_t \cdot \lambda \cdot \theta_t$$

Where:

- η_t : Learning Rate
- λ : Weight Decay Rate is
- θ_t : The current weights of the model are

Why is AdamW so popular today?

- **Better Generalization:**

Models trained with AdamW typically perform better on (new) test data than Adam.

- **More stable training of large models:**

This algorithm is the first choice for training large models such as Transformers, language models (*LLMs* such as *GPT* and *BERT*), and complex deep neural networks.

- **Easier hyperparameter tuning:**

In AdamW, changing the learning rate LR does not negatively affect the optimal value of *Weight Decay* because these two parameters are decoupled.

```
1  optimizer = optimizers.AdamW(  
2      learning_rate=1e-3,  
3      weight_decay=0.01  
4  )  
5
```

g) Nesterov Adaptive Moment Estimation (Nadam):

Nadam (Nesterov accelerated Adaptive Moment Estimation) combines the strengths of Adam which uses adaptive learning rates and momentum with Nesterov momentum which looks ahead to adjust the gradient direction more effectively. By adding this lookahead step Nadam often improves training stability and convergence speed specially for deep neural networks with noisy or sparse gradients.

How work:

1. the gradients (mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

2. the squared gradients (variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

3. It uses bias correction for both:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4. Then, instead of a standard update Nadam applies Nesterov lookahead:

$$\theta_{t+1} = \theta_t - \eta \frac{(\beta_1 \hat{m}_t + \frac{(1 - \beta_1)}{1 - \beta_1^t} g_t)}{\sqrt{\hat{v}_t} + \epsilon}$$

Advantages:

1. **Combines the strengths of Adam and Nesterov:** Nadam combines the strengths of two well known techniques Adam's adaptive moment estimation and Nesterov's accelerated gradient. This blend helps it adapt learning rates

for each parameter individually while also using a lookahead mechanism that anticipates where the parameters will move next.

2. **Better handling of ravines and sharp minima:** Nadam tends to handle sharp minima and narrow ravines more gracefully than standard momentum or Adam alone. In regions of the loss function that change steeply in some directions but are flat in others, typical momentum can overshoot and oscillate.
3. **Can improve generalization for RNNs and sparse gradients:** Nadam is also particularly useful when working with models that have noisy or sparse gradients, such as RNNs or NLP tasks with high dimensional sparse input data. In these situations, the additional stability from Nesterov momentum can make training less erratic and help the optimizer settle into a good minimum more reliably.

Disadvantages:

1. **Performance Gains May Be Task Dependent:** Nadam's performance gains can be task dependent and sometimes marginal which means that for many problems Adam may work just as well without the extra complexity of the Nesterov lookahead.
2. **Adds to computational cost:** Nadam's lookahead step adds a bit more computational overhead compared to Adam. For users working in environments with limited computing resources this extra cost might not justify the small performance benefit.
3. **Hyperparameter Tuning:** Nadam like other adaptive optimizers also relies on good hyperparameter tuning. The optimizer can still become unstable or converge poorly if the learning rate or momentum parameters are not appropriate for the specific problem